

Package ‘colvR’

December 17, 2025

Type Package

Title Poisson Lognormal with Dimension Reduction for Count Matrices
with Missing Data

Version 0.1.1

Description Impute missing data in count tables using a Poisson lognormal model with dimension reduction, with and without zero inflation.

License GPL-3 + file LICENSE

Encoding UTF-8

Language en-US

RoxygenNote 7.3.2

Imports Rcpp, PLNmodels, Matrix, mvtnorm, pbapply, lori, MASS

LinkingTo Rcpp, RcppArmadillo, nloptr

Suggests testthat (>= 3.0.0), knitr, rmarkdown, spelling

Config/testthat/edition 3

VignetteBuilder knitr

SystemRequirements NLOpt, C++17

URL <https://github.com/BBricout/colvR>,
<https://bbricout.github.io/colvR/>

BugReports <https://github.com/BBricout/colvR/issues>

Depends R (>= 3.5)

NeedsCompilation yes

Author Barbara Bricout [aut, cre]

Maintainer Barbara Bricout <barbara.bricout@gmail.com>

Contents

BIC	2
covmat	3
Elbo_grad	4
Elbo_grad_logS	5
fuligule_milouin	6
IC	6
Init	7
Init_ZIP	8
Init_ZIP_q0	9
MatrixToVector	9
Miss.PLNPCA	10
Miss.ZIPLNPCA	12
Miss.ZIPLNPCA.logS	14
Miss.ZIPLNPCA_Steps	15
Miss.ZIPLNPCA_VE	17
PLNPCA_param	19
Predictions	19
Predictions.marginales	20
select_model_bic	21
Simul	22
VectorToMatrix	23
V_theta	24
Index	25

BIC

BIC from ELBO for a ZIP-PLN latent factor model

Description

Computes a Bayesian Information Criterion (BIC)-like score from the Evidence Lower Bound (ELBO) of a fitted model. The penalty accounts for the number of free parameters in a latent factor model with latent dimension q :

$$\text{BIC} = \text{ELBO} - \frac{\log(n)}{2} \left[q\{p - (q - 1)/2\} + p d \right],$$

where n is the number of rows (sites/samples), p the number of columns (species/variables), and d the number of covariates.

Usage

BIC(Y, X, fit, q)

Arguments

<code>Y</code>	Numeric $n \times p$ count matrix.
<code>X</code>	Numeric $n \times d$ design matrix of covariates (first column typically an intercept).
<code>fit</code>	A fitted object (list) containing at least <code>\$elbo</code> , the maximized ELBO value for the model.
<code>q</code>	Integer, size of the latent space (rank).

Details

This criterion mirrors the usual BIC structure but replaces the log-likelihood with the ELBO and uses the parameter count appropriate for a rank- q latent factor structure.

Value

A single numeric value: the BIC score (higher is better in this ELBO-based convention).

See Also

[BIC](#)

covmat	<i>Wrapper for 'lori::covmat'</i>
--------	-----------------------------------

Description

Thin helper that calls `lori::covmat()` without requiring users to attach the **lori** package. It constructs the block-structured covariate matrix X (with $n \times p$ rows) from site-level covariates R , year-level covariates C , and optional site-year covariates E . Any of R , C , or E can be `NULL` to omit that block.

Usage

```
covmat(n, p, R = NULL, C = NULL, E = NULL, center = FALSE)
```

Arguments

<code>n</code>	Integer; number of sites.
<code>p</code>	Integer; number of years.
<code>R</code>	Optional numeric matrix with n rows and d_R columns (site-level covariates). Default: <code>NULL</code> .
<code>C</code>	Optional numeric matrix with p rows and d_C columns (year-level covariates). Default: <code>NULL</code> .
<code>E</code>	Optional numeric matrix with np rows and d_E columns (site-year covariates; one row per site-year observation in the same order as X). Default: <code>NULL</code> .
<code>center</code>	Logical; whether to center columns within each block (as in <code>lori::covmat</code>). Default: <code>FALSE</code> .

Value

A numeric matrix with $n \times p$ rows and $d = d_R + d_C + d_E$ columns (when present), identical to the value returned by `lori::covmat()`.

See Also

[covmat](#)

Examples

```
if (requireNamespace("lori", quietly = TRUE)) {
  set.seed(1)
  n <- 3; p <- 2
  R <- matrix(rnorm(n * 2), nrow = n, ncol = 2) # site-level (d_R = 2)
  C <- matrix(rnorm(p * 2), nrow = p, ncol = 2) # year-level (d_C = 2)
  X <- covmat(n, p, R = R, C = C)
  dim(X) # 6 x 4
}
```

Elbo_grad

ELBO and parameter gradients (ZIP-PLN variational objective)

Description

Computes the Evidence Lower Bound (ELBO) and its gradients with respect to the model and variational parameters for a zero-inflated Poisson log-normal (ZIP-PLN) latent factor model. This is a thin R wrapper around a fast C++ implementation, with a small preprocessing step that replaces NA values in Y by zeros before delegation.

Usage

```
Elbo_grad(data, params, tolxi)
```

Arguments

<code>data</code>	A list with elements: Y Numeric $n \times p$ count matrix (may contain NA). X Numeric $n \times d$ design matrix for the abundance/log-mean part. R Numeric/binary design matrix for the zero-inflation (logit) part (same number of rows as Y); can be NULL if not used.
<code>params</code>	A list with elements: B Matrix of regression coefficients for X (abundance). D Matrix of regression coefficients for X (zero-inflation). C Loadings matrix for the latent factors (rank q). M Variational means of the latent factors. S Variational standard deviations (or log-SDs) of the latent factors.
<code>tolxi</code>	Numeric, convergence tolerance used for the variational bound on the logistic term.

Details

This function sets NA entries of data\$Y to 0 internally (they are not dropped) before calling the C++ backend Elbo_grad_Rcpp. Ensure the dimensions of X, R and the parameter matrices are compatible with Y.

Value

A list containing at least:

elbo Scalar numeric, the ELBO value.

grad_B, grad_D, grad_C, grad_M, grad_S Gradients w.r.t. the corresponding parameters.

See Also

Elbo_grad_Rcpp (backend C++).

Elbo_grad_logS	<i>Calculate the Elbo with log(S) instead of S and the gradients</i>
----------------	--

Description

Calculate the Elbo with log(S) instead of S and the gradients

Usage

```
Elbo_grad_logS(data, params)
```

Arguments

data	list(Y, R, X)
params	list(B, D, C, M, S)

Value

Elbo and gradients

fuligule_milouin	<i>Fuligule milouin example dataset</i>
------------------	---

Description

Small example data for the vignette and examples.

Usage

```
data(fuligule_milouin)
```

Format

A named list with:

Y Count matrix of size $n \times p$.

X Covariate matrix of size $n * p \times d$ (first column may be an intercept).

Details

Dimensions are kept small so that checks and vignettes run quickly.

Source

Internal.

Examples

```
data(fuligule_milouin, package = "colvR")
str(fuligule_milouin, max.level = 1)
```

IC	<i>95% interval for a theta 95% confidence interval for theta</i>
----	---

Description

Computes the 95% confidence interval for theta.

Usage

```
IC(theta, var)
```

Arguments

theta Numeric vector of estimates.

var Numeric vector of associated variances (same length as theta).

Value

A numeric vector of length 2: lower and upper bounds.

Examples

```
IC(1, 0.04)
```

Init

Initialize parameters for a PLN-PCA with missing data

Description

Provides a quick, SVD-based initialization of model and variational parameters from a (possibly sparse) count matrix Y and a design matrix X . The routine fits a (log-transformed) linear model to obtain regression coefficients and uses the SVD of residuals to initialize the latent structure (loadings and variational means).

Usage

```
Init(Y, X, q)
```

Arguments

Y	Numeric $n \times p$ count matrix (may contain NA).
X	Numeric design matrix of covariates. Either $n \times d$ (rowwise design) or $(n \times p) \times d$ (vectorized design matching <code>MatrixToVector(Y)</code>).
q	Integer, target latent dimension (rank) for the factor structure.

Details

Missing values in Y are handled via `na.exclude` in the linear model; residuals at missing entries are then set to zero before the SVD.

Value

A list with elements:

- B Matrix of regression coefficients (dimensions follow the chosen design; typically $(1 \times d)$ in the rowwise case).
- C Loadings matrix $(p \times q)$ for the latent factors.
- M Variational means of the latent factors $(n \times q)$.
- S Variational standard deviations (initialized to a small constant; $n \times q$).

See Also

[MatrixToVector](#), [VectorToMatrix](#)

Examples

```
set.seed(1)
n <- 30; p <- 8; d <- 2; q <- 2
Y <- matrix(rpois(n * p, 2), n, p)
X <- cbind(1, rnorm(n))      # rowwise design
init <- Init(Y, X, q)
str(init)
```

Init_ZIP

Initialize parameters for the ZIP case (zero-inflated)

Description

SVD-based initialization of parameters for a zero-inflated Poisson log-normal latent factor model (ZIP-PLN). Regression coefficients for the abundance part (B) are obtained by regressing $\log(1 + \text{vec}(Y))$ on X; zero-inflation coefficients (D) come from a logistic regression of $I(Y > 0)$ on X. The latent structure is initialized from the SVD of the residual matrix.

Usage

```
Init_ZIP(Y, X, q)
```

Arguments

Y	Numeric n x p count matrix (may contain NA).
X	Numeric design matrix with n*p rows and d columns, aligned with $\text{vec}(Y)$ (column-wise vectorization).
q	Integer, target latent dimension (rank).

Value

A list with elements:

- B Abundance (Poisson) regression coefficients (1 x d).
- D Zero-inflation (logit) regression coefficients (1 x d).
- C Loadings matrix (p x q).
- M Variational means of latent factors (n x q).
- S Variational scale parameters.

See Also

[Init](#) for the non-ZI initializer; [MatrixToVector](#), [VectorToMatrix](#)

Examples

```
set.seed(1)
n <- 30; p <- 10; d <- 3; q <- 2
Y <- matrix(rpois(n*p, 2), n, p)
X <- cbind(1, rnorm(n*p), rnorm(n*p)) # (n*p) x d, vectorized design
init <- Init_ZIP(Y, X, q)
str(init)
```

Init_ZIP_q0

*Init_ZIP_q0***Description**

Parameters initialisation in the zero inflated case without the latent factors

Usage

```
Init_ZIP_q0(Y, X, q)
```

Arguments

Y	count matrix
X	covariates
q	size of the latent space

Value

A list with elements:

B Matrix of Poisson regression coefficients (1 x d).

D Matrix of logistic regression coefficients (1 x d).

MatrixToVector

*MatrixToVector***Description**

Convert a numeric matrix to a numeric vector (column-major order). Implemented in C++ via Rcpp/Armadillo.

Usage

```
MatrixToVector(matrix)
```

Arguments

`matrix` Numeric matrix (n x p).

Value

Numeric vector of length n*p.

See Also

[VectorToMatrix](#)

Miss.PLNPCA

PLN-PCA with missing data (variational fit + imputation)

Description

Fits a Poisson log-normal PCA model (PLN-PCA) to a count matrix with missing values, using a variational objective optimized by NLOpt-based routines. The function estimates model/variational parameters and returns expected counts to impute missing entries.

Usage

```
Miss.PLNPCA(Y, X, O = NULL, w = NULL, q, params = NULL, config = NULL)
```

Arguments

`Y` Numeric n x p count matrix. May contain NA.

`X` Numeric design matrix of covariates: either n x d (rowwise) or (n*p) x d (vectorized to match `MatrixToVector(Y)`).

`O` Optional numeric n x p matrix of offsets (default: zeros).

`w` Optional numeric vector of length n with observation weights (default: all ones).

`q` Integer, latent rank (dimension of the latent space).

`params` Optional initial parameters (list) typically produced by [Init](#); if NULL, they are initialized internally.

`config` Optional optimizer configuration; if NULL, defaults to `PLNPCA_param()$config_optim`.

Details

Two designs for X are supported:

1. **Rowwise design** (`nrow(X) == n`): one row per sample.
2. **Vectorized design** (`nrow(X) == n*p`): X aligns with `vec(Y)` (vectorization by columns).

Value

A list with components:

`mStep` List with `beta` (1 x d) and `C` (p x q).

`eStep` List with `M` (n x q) and `S` (n x q).

`pred` List with `A` and `predicted`, both equal to the n x p matrix of expected counts used for imputation:

$$A = \exp(O + XB + MC^\top + 0.5(S \odot S)(C \odot C)^\top),$$

where $\odot$ denotes the Hadamard product and XB is reshaped to n x p when X is vectorized.

`iter` Integer, number of iterations.

`elboPath` Numeric vector of objective (ELBO) values over iterations.

`elbo` Final ELBO value.

`params.init` The initial parameters used.

`monitoring` Optimizer log/diagnostics as returned by the backend.

See Also

[Init](#), [PLNPCA_param](#), [MatrixToVector](#), [VectorToMatrix](#), [Elbo_grad](#)

Examples

```
## Not run:
set.seed(1)
n <- 40; p <- 12; d <- 2; q <- 2
Y <- matrix(rpois(n*p, 2), n, p)
Y[sample(length(Y), 20)] <- NA
X <- cbind(1, rnorm(n))      # rowwise design

fit <- Miss.PLNPCA(
  Y = Y, X = X, q = q,
  O = matrix(0, n, p),
  w = rep(1, n),
  params = NULL,
  config = NULL
)
str(fit$pred$A)      # expected counts (n x p)
fit$elbo             # final ELBO

## End(Not run)
```

Miss.ZIPLNPCA

*ZI-PLN-PCA with missing data (variational fit + imputation)***Description**

Fits a zero-inflated Poisson log-normal (ZI-PLN-PCA) latent factor model to a count matrix with missing values. The routine optimizes a variational objective with NLOpt backends and returns parameter estimates, variational parameters and an imputed matrix.

Usage

```
Miss.ZIPLNPCA(Y, X, q, params = NULL, config = NULL, tolS = NULL, tolxi = NULL)
```

Arguments

Y	Numeric $n \times p$ count matrix. May contain NA.
X	Numeric design matrix with $n \times p$ rows and d columns, aligned with <code>vec(Y)</code> (column-wise vectorization).
q	Integer, latent rank (dimension of the latent space).
params	Optional list of initial parameters. If NULL, they are initialized internally via <code>Init_ZIP_q0</code> (when $q = 0$) or <code>Init_ZIP</code> (when $q > 0$).
config	Optional list of optimizer controls. If NULL, a default configuration is used (NLOpt backend, MMA algorithm, sensible tolerances).
tolS	List with numeric bounds for the variational scales S : elements lower and upper. Defaults to <code>list(lower = 0, upper = 1)</code> .
tolxi	Numeric tolerance for the Jaakkola-type ξ updates in the logistic bound (default $1e-4$).

Details

This function supports latent rank $q \geq 0$. When $q = 0$, it reduces to a ZIP regression (no latent factors). When $q > 0$, it fits a ZIP-PLN factor model.

Internally, a binary mask $R = 1_{\{\text{observed}\}}(Y)$ is created; a copied matrix `Y.na` sets missing entries to zero for objective evaluation. The optimizer bounds for S are taken from `tolS`. Ensure that the vectorization and parameter stacking used in the optimizer are consistent with the shapes listed above.

Value

A list with components:

`mStep` List of model parameters. For $q = 0$: γ ($d \times 1$) and β ($d \times 1$). For $q > 0$: same plus the loading matrix C ($p \times q$).

`eStep` List of variational parameters. For $q = 0$: only ξ ($n \times p$). For $q > 0$: M ($n \times q$), S ($n \times q$) and ξ ($n \times p$).

`pred` List with predictors and expected counts: `mu` ($n \times p$, abundance mean), `nu` ($n \times p$, zero-inflation mean), `A` ($n \times p$, PLN expectation), `predicted` ($n \times p$, predicted values). For $q = 0$, `predicted` = $\exp(XB)$; for $q > 0$,

$$\text{predicted} = \exp(XB + MC^T + 0.5(S \odot S)(C \odot C)^T).$$

`imputed` $n \times p$ matrix equal to `xi * A` at missing entries of `Y`, and `Y` elsewhere (ZIP expectation).

`iter` Integer, number of iterations.

`elboPath` Numeric vector of objective (ELBO) values over iterations.

`elbo` Final ELBO value.

`params.init` Parameters as recorded from the backend call.

`monitoring` Optimizer diagnostics/logs from the backend.

`gradB`, `gradD`, `gradC`, `gradM`, `gradS` Gradients of the ELBO with respect to the corresponding parameters, obtained via [Elbo_grad](#).

See Also

[Init_ZIP](#), [Init_ZIP_q0](#), [Elbo_grad](#), [VectorToMatrix](#)

Examples

```
## Not run:
set.seed(1)
n <- 40; p <- 12; d <- 3
q <- 2
Y <- matrix(rpois(n*p, 2), n, p)
Y[sample(length(Y), 25)] <- NA
# Vectorized design (n*p) x d:
X <- cbind(1, rnorm(n*p), rnorm(n*p))

fit <- Miss.ZIPLNPCA(Y = Y, X = X, q = q)
str(fit$mStep)
str(fit$eStep)
image(log1p(fit$imputed)) # quick look at imputed counts

# Rank-0 (ZIP regression without latent factors):
fit0 <- Miss.ZIPLNPCA(Y = Y, X = X, q = 0)
fit0$eStep$xi[1:3, 1:3]

## End(Not run)
```

Miss.ZIPLNPCA.logS	<i>ZI-PLN-PCA with missing data using $\log(S)$ parametrization</i>
--------------------	--

Description

Fits a zero-inflated Poisson log-normal (ZI-PLN-PCA) latent factor model to a count matrix with missing values, using a variational objective and an **alternative parametrization** on the variational scales:

$$\log(S) \text{ instead of } S.$$

This guarantees positivity of $\log(S)$ and allows simple box constraints on $\log(S)$ via `tolLogS`.

Usage

```
Miss.ZIPLNPCA.logS(
  Y,
  X,
  q,
  params = NULL,
  config = NULL,
  tolxi = NULL,
  tolLogS = NULL
)
```

Arguments

<code>Y</code>	Numeric $n \times p$ count matrix. May contain NA.
<code>X</code>	Numeric design matrix with $n \times p$ rows and d columns, aligned with <code>vec(Y)</code> (vectorization by columns).
<code>q</code>	Integer, latent rank (dimension of the latent space).
<code>params</code>	Optional list of initial parameters. If NULL, they are initialized by <code>Init_ZIP</code> . If provided, it must include a positive S ; the function internally adds <code>logS <- log(S)</code> .
<code>config</code>	Optional list of optimizer controls. If NULL, defaults to <code>PLNPCA_param()\$config_optim</code> .
<code>tolxi</code>	Numeric tolerance for the Jaakkola-type ξ updates in the logistic bound (default $1e-4$).
<code>tolLogS</code>	Numeric upper bound applied to <code>logS</code> (box constraint). Defaults to <code>Inf</code> . Use this to avoid excessively large variational scales when needed.

Details

Missing entries are handled by a mask $R = 1_{\{\text{observed}\}}(Y)$; a working matrix `Y.na` sets missings to 0 for the objective evaluation. Box constraints are set on `logS` via `config$upper_bounds`, using `tolLogS`. Make sure `X` matches `vec(Y)`.

Value

A list with components:

`mStep` Model parameters: γ ($d \times 1$), β ($d \times 1$), and loadings C ($p \times q$).

`eStep` Variational parameters: means M ($n \times q$), scales S ($n \times q$), and logistic bound parameters ξ ($n \times p$).

`pred` List with predictors and expected counts: μ ($n \times p$), ν ($n \times p$), backend Poisson mean A ($n \times p$), and predicted recomputed in R:

$$\exp(XB + MC^T + 0.5(S \odot S)(C \odot C)^T).$$

`imputed` $n \times p$ matrix equal to $\xi * A$ at missing entries of Y , and Y elsewhere.

`iter` Number of iterations; `elboPath` (trajectory of ELBO); `elbo` (final ELBO).

`params.init` Initial parameters passed to the backend (with `logS` added).

`monitoring` Optimizer diagnostics/logs.

`gradB`, `gradD`, `gradC`, `gradM`, `gradS` ELBO gradients w.r.t. corresponding parameters (via [Elbo_grad](#)).

See Also

[Miss.ZIPLNPCA](#) for the S parametrization, [Init_ZIP](#), [Elbo_grad](#)

Examples

```
## Not run:
set.seed(1)
n <- 40; p <- 12; d <- 3; q <- 2
Y <- matrix(rpois(n*p, 2), n, p); Y[sample(length(Y), 20)] <- NA
X <- cbind(1, rnorm(n*p), rnorm(n*p)) # (n*p) x d

fit <- Miss.ZIPLNPCA.logS(Y = Y, X = X, q = q, tolLogS = 2) # cap logS
fit$elbo
str(fit$eStep$S)

## End(Not run)
```

Miss.ZIPLNPCA_Steps	ZI-PLN-PCA (missing data) with parameterwise optimization steps
---------------------	---

Description

Fits a zero-inflated Poisson log-normal (ZI-PLN-PCA) latent factor model to a count matrix with missing values using a **blockwise** optimization strategy. At each outer iteration, parameters are updated by activating one block among $\{B, D, C, M, S\}$ while keeping the others fixed, via an NLOpt backend. The loop stops when both parameter and ELBO changes are below thresholds or when `maxiter` is reached.

Usage

```
Miss.ZIPLNPCA_Steps(Y, X, q, params = NULL, config_vem = NULL, config = NULL)
```

Arguments

Y	Numeric $n \times p$ count matrix. May contain NA.
X	Numeric design matrix with $n \times p$ rows and d columns, aligned with <code>vec(Y)</code> (column-wise vectorization).
q	Integer, latent rank (dimension of the latent space).
params	Optional list of initial parameters. If NULL, <code>Init_ZIP</code> is used.
config_vem	List of controls for the outer blockwise loop (variational EM-like), with fields: <code>maxiter</code> Maximum number of outer iterations (default 1e6). <code>ftol</code> ELBO tolerance; stop if $ \text{ELBO}_{\{t\}} - \text{ELBO}_{\{t-1\}} \leq \text{ftol}$. <code>xtol</code> Parameter tolerance; stop if $\max \theta_{\{t\}} - \theta_{\{t-1\}} \leq \text{xtol}$. <code>tolS</code> List with lower, upper bounds for S. <code>tolxi</code> Tolerance used in the Jaakkola-type ξ updates for the logistic bound.
config	List of controls for the NLOpt backend (per-block inner solve), e.g. <code>algorithm</code> , <code>maxeval</code> , <code>ftol_abs</code> , <code>xtol_abs</code> , <code>ftol_rel</code> , <code>xtol_rel</code> , <code>trace</code> , <code>maxtime</code> , as well as <code>upper_bounds</code> , <code>lower_bounds</code> (filled here from <code>tolS</code> and dimensions).

Details

Missing entries are handled through a binary mask and a working copy of Y with zeros at missing locations for the objective evaluation.

A binary mask $R = 1_{\{\text{observed}\}}(Y)$ is built; a working matrix $Y.na$ replaces missings by 0 for the objective. Box constraints for S are set using `config_vem$tolS` and injected into `config$upper_bounds` and `config$lower_bounds`. Each block update calls the backend `nlopt_optimize_ZIP_Steps(data, params, config, tolxi, active_blocks)` with a single active block at a time.

Predicted mean: in this implementation, the Poisson mean used for the R-side recomputation is

$$\exp(\mu + MC^\top + 0.5 S (C \odot C)^\top),$$

where $\mu = XB$ reshaped into $n \times p$. Note that other functions in the package employ $0.5 (S \odot S) (C \odot C)^\top$. Vérifiez que la paramétrisation voulue est cohérente avec le backend.

Value

A list with components:

`mStep` Model parameters: γ ($d \times 1$), β ($d \times 1$), loadings C ($p \times q$).

`eStep` Variational parameters: M ($n \times q$), S ($n \times q$), logistic bound parameters ξ ($n \times p$).

`pred` List with predictors and expected counts: μ ($n \times p$), ν ($n \times p$), backend mean A ($n \times p$), and predicted recomputed in R.

`imputed` $n \times p$ matrix equal to $\xi * A$ at missing entries of Y, and Y elsewhere.

`iter` Number of outer iterations performed.

`elboPath` Numeric vector of ELBO values across outer iterations.
`elbo` Final ELBO value.
`params.init` Parameters used for initialization.
`monitoring` List with status (stopping reason) and iterations.
`gradB, gradD, gradC, gradM, gradS` ELBO gradients w.r.t. each parameter block (from [Elbo_grad](#)) at the solution.

See Also

[Miss.ZIPLNPCA](#), [Miss.ZIPLNPCA.logS](#), [Init_ZIP](#), [Elbo_grad](#)

Examples

```
## Not run:
set.seed(1)
n <- 50; p <- 15; d <- 3; q <- 2
Y <- matrix(rpois(n*p, 2), n, p); Y[sample(length(Y), 30)] <- NA
X <- cbind(1, rnorm(n*p), rnorm(n*p)) # (n*p) x d, vectorized design

fit <- Miss.ZIPLNPCA_Steps(Y, X, q)
fit$elbo
plot(fit$elboPath, type = "l", xlab = "outer iter", ylab = "ELBO")
str(fit$mStep); str(fit$eStep)

## End(Not run)
```

Miss.ZIPLNPCA_VE	<i>ZI-PLN-PCA (missing data) — variational E-step solver</i>
------------------	--

Description

Fits a zero-inflated Poisson log-normal (ZI-PLN-PCA) latent factor model on a count matrix with missing values using a single call to a variational optimization backend (VE). Returns model/variational parameters and expected Poisson means suitable for imputation and prediction.

Usage

```
Miss.ZIPLNPCA_VE(Y, X, q, params = NULL, config_vem = NULL, config = NULL)
```

Arguments

<code>Y</code>	Numeric $n \times p$ count matrix. May contain NA.
<code>X</code>	Numeric design matrix with $n \times p$ rows and d columns, aligned with <code>vec(Y)</code> (vectorization by columns).
<code>q</code>	Integer, latent rank (dimension of the latent space).

params	Optional list of initial parameters. If NULL, Init_ZIP is called.
config_vem	List of outer VE controls (stopping and bounds), with fields: - maxiter Maximum iterations (not always used by this backend, default 10000). - ftol ELBO tolerance (default 1e-08). - xtol Parameter tolerance (default 1e-04). - tolS List with lower, upper bounds for S (default list(lower = 1e-4, upper = 1)). - tolxi Tolerance for the Jaakkola-type ξ updates (default 1e-04).
config	List of NLOpt controls for the backend (algorithm, tolerances, maxeval, etc.). If NULL, sensible defaults are provided.

Details

Missing entries are handled via a binary mask $R = 1_{\{\text{observed}\}}(Y)$, and a working copy $Y.na$ with zeros at missing locations for the objective evaluation. Box constraints on S are injected into `config$lower_bounds` and `config$upper_bounds` using `config_vem$tolS`.

Predicted mean: ici, la recomposition côté R utilise

$$\exp(XB + MC^{\top} + 0.5 S (C \odot C)^{\top}).$$

D'autres fonctions du package emploient $0.5 (S \odot S) (C \odot C)^{\top}$. Vérifie et harmonise selon la paramétrisation attendue par ton backend.

Value

A list with components:

mStep Model parameters: gamma (d x 1), beta (d x 1), loadings C (p x q).

eStep Variational parameters: M (n x q), S (n x q), xi (n x p).

pred List with predictors and expected counts: mu (n x p), nu (n x p), backend mean A (n x p), and predicted recomputed in R.

elbo Final ELBO value.

params.init Initial parameters used.

monitoring Backend diagnostics (if provided).

elbo1, elbo2, elbo3, elbo4, elbo5 Decomposition terms of the ELBO (as returned by the backend).

See Also

[Miss.ZIPLNPCA](#), [Miss.ZIPLNPCA_Steps](#), [Init_ZIP](#)

Examples

```
## Not run:
set.seed(1)
n <- 40; p <- 12; d <- 3; q <- 2
Y <- matrix(rpois(n*p, 2), n, p); Y[sample(length(Y), 20)] <- NA
X <- cbind(1, rnorm(n*p), rnorm(n*p)) # (n*p) x d

fit <- Miss.ZIPLNPCA_VE(Y = Y, X = X, q = q)
fit$elbo
str(fit$mStep); str(fit$eStep)

## End(Not run)
```

PLNPCA_param	<i>PLNPCA_param</i>
--------------	---------------------

Description

Internal helper for PLNPCA parameters.

Value

Invisibly, nothing.

Predictions	<i>Predictive simulations and intervals</i>
-------------	---

Description

Draws parameter samples using the sandwich variance V_{θ} , runs VE updates, and simulates predictive counts with zero-inflation. Returns per-cell prediction intervals and coverage.

Usage

```
Predictions(Y, X, fit, MC)
```

Arguments

- Y An $n \times p$ count matrix (NAs allowed).
- X An $(n \times p) \times d$ design matrix (stacked by blocks of size n).
- fit Estimator output list.
- MC Integer, number of Monte Carlo draws.

Value

A list with elements:

- Z, U, Y.hat, pred: MC samples
- xi: VE xi matrices for each draw
- lower, upper: 2.5% and 97.5% predictive quantiles
- level: average empirical coverage of [lower, upper]

Predictions.marginales

Prediction intervals (marginal) for ZIPLNPCA-like model

Description

Computes cell-wise predictive simulations and 95 under the **marginal** model (integrating latent factors).

Usage

```
Predictions.marginales(Y, X, fit, MC)
```

Arguments

Y	Matrix n x p of observed counts.
X	Design matrix of shape (n*p) x d (your current design), or any object compatible with <code>as.numeric(X %*% beta)</code> producing a vector of length n*p later reshaped into n x p.
fit	A fitted object providing <code>fit\$mStep\$beta</code> , <code>fit\$mStep\$gamma</code> , <code>fit\$mStep\$C</code> , and <code>fit\$eStep\$M</code> , <code>fit\$eStep\$S</code> , <code>fit\$eStep\$xi</code> . Only <code>beta</code> , <code>gamma</code> , <code>C</code> are used here.
MC	Integer, number of Monte Carlo draws.

Details

The parameter vector is sampled with the sandwich variance from `V_theta(Y, X, fit)`. The ordering assumed for `theta` is `c(vec(gamma), vec(beta), vec(C))`, consistent with your code.

Zero-inflation convention used here:

$$\pi_0 = \text{logit}^{-1}(X\gamma)$$

is the probability of a structural zero. We simulate $U = \mathbf{1}\{\text{keep Poisson}\} \sim \text{Bernoulli}(1 - \pi_0)$, and the final prediction is $Y = U \cdot Y_{\text{Poisson}}$.

Value

A list with elements:

- Z: list of length MC with Gaussian latent draws (n x p).
- U: list of length MC with Bernoulli “keep” masks (n x p).
- Y.hat: list of length MC with Poisson draws (n x p).
- pred: list of length MC with final ZIP predictions (n x p).
- lower, upper: matrices n x p with 2.5
- level: scalar, empirical coverage vs Y (if used as “truth”).

See Also

[V_theta](#), [VectorToMatrix](#)

Examples

```
## Not run:
n <- 5; p <- 3; d <- 2
set.seed(1)
Y <- matrix(rpois(n*p, 2), n, p)
X <- matrix(rnorm(n*p*d), n*p, d)
fit <- list(
  mStep = list(beta = rnorm(d), gamma = rnorm(d), C = matrix(rnorm(p*2), p, 2)),
  eStep = list(M = matrix(0, n, 2), S = matrix(1, n, 2), xi = matrix(0.5, n, p))
)
out <- Predictions.marginales(Y, X, fit, MC = 100)
str(out$lower)

## End(Not run)
```

select_model_bic

Model selection by BIC over candidate latent dimensions

Description

For each value in latents, fits a model via `fun(Y, X, q)` and computes the corresponding BIC. Returns the vector of BIC values in the same order as latents.

Usage

```
select_model_bic(Y, X, fun, latents)
```

Arguments

Y	Response matrix (n x p).
X	Covariate matrix (n x d).
fun	Fitting function called as fun(Y, X, q) that returns a fit object.
latents	Integer/numeric vector of candidate latent dimensions.

Details

A progress bar is displayed via **pbapply**. The BIC function used must be available in your namespace (e.g., colvR::BIC if you define one, or stats::BIC if you call it explicitly).

Value

A numeric vector of BIC values, one per element of latents, in the same order as latents.

Simul	<i>Simulate zero-inflated Poisson log-normal PCA data</i>
-------	---

Description

Generates simulated observations Y under a ZIP-LN-PCA structure from parameters in theta and dimensions in dim.

Usage

```
Simul(X, theta, dim)
```

Arguments

X	An $(n \times p) \cdot d$ design matrix stacked so that VectorToMatrix($X \%*\% B$, n, p) produces the mean matrix mu.
theta	List of model parameters with components: • B: coefficient matrix (d x 1 or compatible) • D: zero-inflation coefficient matrix (d x 1 or compatible) • C: loading matrix (p x q)
dim	List of dimensions with entries n, p, d, q.

Details

Relies on VectorToMatrix() being available in the package.

Value

A list with elements:

- Y: simulated counts (n x p)
- W: latent scores (n x q)
- U: zero-inflation indicators (n x p)
- Z: Poisson draws before zero-inflation (n x p)
- Lambda: Poisson intensities (n x p)
- mu: mean matrix (n x p)

Examples

```
## Not run:
n <- 5; p <- 4; d <- 3; q <- 2
X <- matrix(rnorm(n*p*d), n*p, d)
theta <- list(B = matrix(rnorm(d)), D = matrix(rnorm(d)), C = matrix(rnorm(p*q), p, q))
dim <- list(n = n, p = p, d = d, q = q)
out <- Simul(X, theta, dim)
str(out)

## End(Not run)
```

VectorToMatrix

VectorToMatrix

Description

Convert a numeric vector to a numeric matrix (column-major order). Implemented in C++ via Rcpp/Armadillo.

Usage

```
VectorToMatrix(vector, n, p)
```

Arguments

vector	Numeric vector of length n*p.
n	Integer, number of rows.
p	Integer, number of columns.

Value

Numeric matrix (n x p).

See Also

[MatrixToVector](#)

V_theta	<i>Variance-covariance matrix of theta</i>
---------	--

Description

Computes $V(\hat{\theta})$ using a sandwich (robust) variance formula.

Usage

```
V_theta(Y, X, fit, ginv_tol = 1e-12, verbose = TRUE)
```

Arguments

Y	An n x p matrix of counts.
X	An (n*p) x d design matrix, stacked by blocks of size n.
fit	Output list returned by the estimator.
ginv_tol	Numeric tolerance used by MASS::ginv() when a pseudo-inverse is needed.
verbose	Logical; if TRUE, prints diagnostic messages when solve() fails or is unstable.

Value

A symmetric variance-covariance matrix for theta.

Index

- * **datasets**
 - fuligule_milouin, [6](#)
- * **internal**
 - MatrixToVector, [9](#)
 - PLNPCA_param, [19](#)
 - VectorToMatrix, [23](#)
- BIC, [2](#), [3](#)
- covmat, [3](#), [4](#)
- Elbo_grad, [4](#), [11](#), [13](#), [15](#), [17](#)
- Elbo_grad_logS, [5](#)
- fuligule_milouin, [6](#)
- IC, [6](#)
- Init, [7](#), [8](#), [10](#), [11](#)
- Init_ZIP, [8](#), [12–18](#)
- Init_ZIP_q0, [9](#), [12](#), [13](#)
- MatrixToVector, [7](#), [8](#), [9](#), [11](#), [23](#)
- Miss.PLNPCA, [10](#)
- Miss.ZIPLNPCA, [12](#), [15](#), [17](#), [18](#)
- Miss.ZIPLNPCA.logS, [14](#), [17](#)
- Miss.ZIPLNPCA_Steps, [15](#), [18](#)
- Miss.ZIPLNPCA_VE, [17](#)
- PLNPCA_param, [11](#), [19](#)
- Predictions, [19](#)
- Predictions.marginales, [20](#)
- select_model_bic, [21](#)
- Simul, [22](#)
- V_theta, [21](#), [24](#)
- VectorToMatrix, [7](#), [8](#), [10](#), [11](#), [13](#), [21](#), [23](#)